Project Assignment – Metrics & Visual Flow

UC Leuven-Limburg

2026-2027

Group Name: F2	**Date:** 19/10/2026
Team Members: BELABED Lina, ROTTÉ Nand, NDACASABA Eddy	**Project:** Food Dataset / Dashboard

Table of Contents

Phase 1 – User Questions 1
Phase 2 – Define the features 4
Explanation of our “why” 5
Phase 3 – Design the Reactive Graph 6
1. Sources (Where Changes Start) 6
2. Conductors (Intermediate, cached, reused) 6
3. Endpoints (Where Change Ends) 7
The Mermaid Diagram 8
Reactive Graph Design & Rationale 8

Phase 1 – User Questions

Before designing the app, we must define the exact questions the users will ask while working. These questions bridge the gap between the broad business problem and the specific app features. Each question contains a variable part in **bold** (which dictates the interactivity) and is paired with an Acceptance Criterion (**Given**/**When**/**Then**) to ensure the feature is testable.

#	User question, variable parts in bold	Type	Acceptance criterion
Q1	How do newly scanned products compare to their top suggested matches based on name, category?	☑ Descriptive ☐ Diagnostic ☐ Predictive ☐ What-if **Story:** S1	**Given** an unvalidated product is selected, **When** the matching algorithm runs, **Then** a side-by-side comparison table displays the candidate products ranked by total similarity score with highlighted field differences.
Q2	Which unvalidated products fall within the **selected match confidence range**?	☑ Descriptive ☐ Diagnostic ☐ Predictive ☐ What-if **Story:** S2	**Given** the user sets a confidence filter slider (e.g., 85%–100%), **When** the filter is applied, **Then** the queue displays only unvalidated items meeting that score threshold.
Q3	Which micronutrients fields are missing from the **selected unvalidated product** and what values does it match?	☑ Descriptive ☐ Diagnostic ☐ Predictive ☐ What-if **Story:** S3	**Given** a candidate product is paired with an unvalidated item, **When** the user clicks "Fill Missing Data", **Then** empty macro fields on the target record are populated with values from the candidate record.

Q4	How do predicted yield, FP rate and manual review workload change if the **text weight** and **numeric weight** are adjusted?	☐ Descriptive ☐ Diagnostic ☐ Predictive ☑ What-if **Story:** S8	**Given** the Developer adjusts the text and numeric weight sliders in the simulator **When** the simulation is executed **Then** the dashboard displays predicted yield, false-positive estimates, and manual review workload.
Q5	Which food categories have the largest backlog of **unvalidated products**?	☑ Descriptive ☐ Diagnostic ☐ Predictive ☐ What-if **Story:** S6	**Given** the database manager opens the backlog overview, **When** the category counts are calculated, **Then** a chart displays the validated vs. unvalidated product counts ranked by backlog size.
Q6	How can I locate a specific product by its **barcode** or **brand name**?	☑ Descriptive ☐ Diagnostic ☐ Predictive ☐ What-if **Story:** S5	**Given** the dietitian enters a barcode or brand name into the search bar, **When** the search is submitted, **Then** the matching active and unvalidated products are displayed for manual inspection.
Q7	How often are duplicate suggestions rejected by the dietitian for **food category**?	☐ Descriptive ☑ Diagnostic ☐ Predictive ☐ What-if **Story:** S4	**Given** the dietitian clicks "Reject duplicate", **When** the rejection is saved, **Then** the product is marked as "not a duplicate" and the rejection count for that category updates.

Q8	What are the monthly **duplicate merge rates** and **data completeness** metrics?	☑ Descriptive ☐ Diagnostic ☐ Predictive ☐ What-if **Story:** S7	**Given** the database manager selects a specific month, **When** the report is generated, **Then** the summary metrics for merge rates and completeness are displayed and ready for export.

Phase 2 – Define the features

Here, we translate the abstract user questions into concrete UI elements. For every question, we identify:

- **The Inputs:** what the user interacts with,
- **App Computes:** what the app must calculate behind the scenes,
- **The Views:** what the user looks at to get their answer.

This ensures every visual element has a clear purpose.

#	User Controls (Inputs)	App Computes	User Sees (Views)	Linked to
Q1	**Table row selection** ⇒ Click on an unvalidated product	Runs similarity algorithm on the selected product's text and numeric data	Side-by-side **comparison table** showing the target product vs. top active matches, with field differences highlighted	S1
Q2	**Slider** ⇒ Match confidence range, e.g., 85% - 100%	Filters the list of unvalidated products based on their top match score	Filtered **Queue Table** of unvalidated products	S2
Q3	**Action Button** ⇒ "Fill Missing Data"	Copies missing numerical values from the selected candidate product to the unvalidated product	Updated **target record** / confirmation message	S3
Q4	**Sliders** ⇒ Text weight, Numeric weight **Action Button** ⇒ "Simulate"	Calculates the predicted yield, false positive (FP) rate, and manual review workload	**Before/after line chart** or bar chart showing the impact of the new weights	S8
Q5	**Dropdown** ⇒ Select food category	Counts validated vs. unvalidated products per category	Sorted **Bar Chart** showing the backlog size per category	S6
Q6	**Text Search Box** ⇒ Barcode or Brand name	Filters the entire database for exact/partial matches	Filtered **Search Results Table**	S5
Q7	**Dropdown** ⇒ Select category	Counts how many times the dietitian clicked "Reject" on a duplicate suggestion	**Bar Chart** or summary table showing rejected suggestions	S4
Q8	**Date Picker** ⇒ Select Month	Aggregates merge rates and data completeness for that month	**KPI cards** / Summary text + **Export Button**	S7

Explanation of our “why”

- **Two kinds of inputs:** We used both standard widgets (sliders, dropdowns, text boxes) and **plot/table events**. For example, in Q1, clicking a row in the data table acts as an input that triggers the comparison view.
- **Choosing the view from the question:**
    - For comparing relationships (Q1) and viewing individual records (Q2, Q6), we chose **Data Tables**.
    - For comparing values across categories (Q5, Q7), we chose **Sorted Bar Charts**.
    - For scenario results (Q4), we chose a **Before/After chart** to clearly show the impact of the weight sliders.
- **Traceability:** Every feature traces back to a user story. Q1-Q3 focus on the dietitian's immediate tasks, Q4 focuses on the developer's need to tune the system, and Q5-Q8 focus on the database manager's oversight.

Phase 3 – Design the Reactive Graph

A Shiny app is not just a static dashboard; it is a flow of data. To prevent bugs and performance issues, like the app freezing when a slider moves for example, we mapped out the reactive logic. We categorize every element into 3 categories:

- **Sources:** where change starts,
- **Conductors:** cached calculations that are computed once and reused,
- **Endpoints:** where the change ends, like a chart updating.

This enforces the *"compute once, use many times"* rule.

1. Sources (Where Changes Start)

These are the inputs from the user, or state that survives a redraw.

- `input$confidence_slider` (Q2)
- `input$table_rows_selected` (Q1)
- `input$fill_btn` (Q3)
- `input$simulate_btn`, `input$text_weight`, `input$numeric_weight` (Q4)
- `input$search_query` (Q6)
- `input$category_filter` (Q5, Q7)
- `input$month_picker` (Q8)
- **State:** `selected_product`

  ⇒ We need a `reactiveVal()` to remember which product was clicked. If we don't use state, the selection disappears when the table redraws.

2. Conductors (Intermediate, cached, reused)

These are the `reactive()` and `eventReactive()` functions.

- **`filtered_queue()`:** A `reactive()` that filters the unvalidated products based on input$confidence_slider. We reuse this for Q2's table.
- **`similarity_results()`:** A `reactive()` that runs the matching algorithm whenever `selected_product()` changes. It reads the text and numeric datasets and outputs the top matches. (Used for Q1).
- **`simulation_results()`:** An `eventReactive()` triggered by `input$simulate_btn`. The role we applied here is *"Separate cheap and expensive work"*. We don't run the simulation every time a slider moves; only when the user clicks "Simulate" (Used for Q4).
- **`search_results()`:** A `reactive()` that searches the database based on input$search_query (Used for Q6).
- **`backlog_stats()`:** A `reactive()` that groups the data by category (Used for Q5 and Q7).

3. Endpoints (Where Change Ends)

These are your `render...()` outputs and `observeEvent()` side effects.

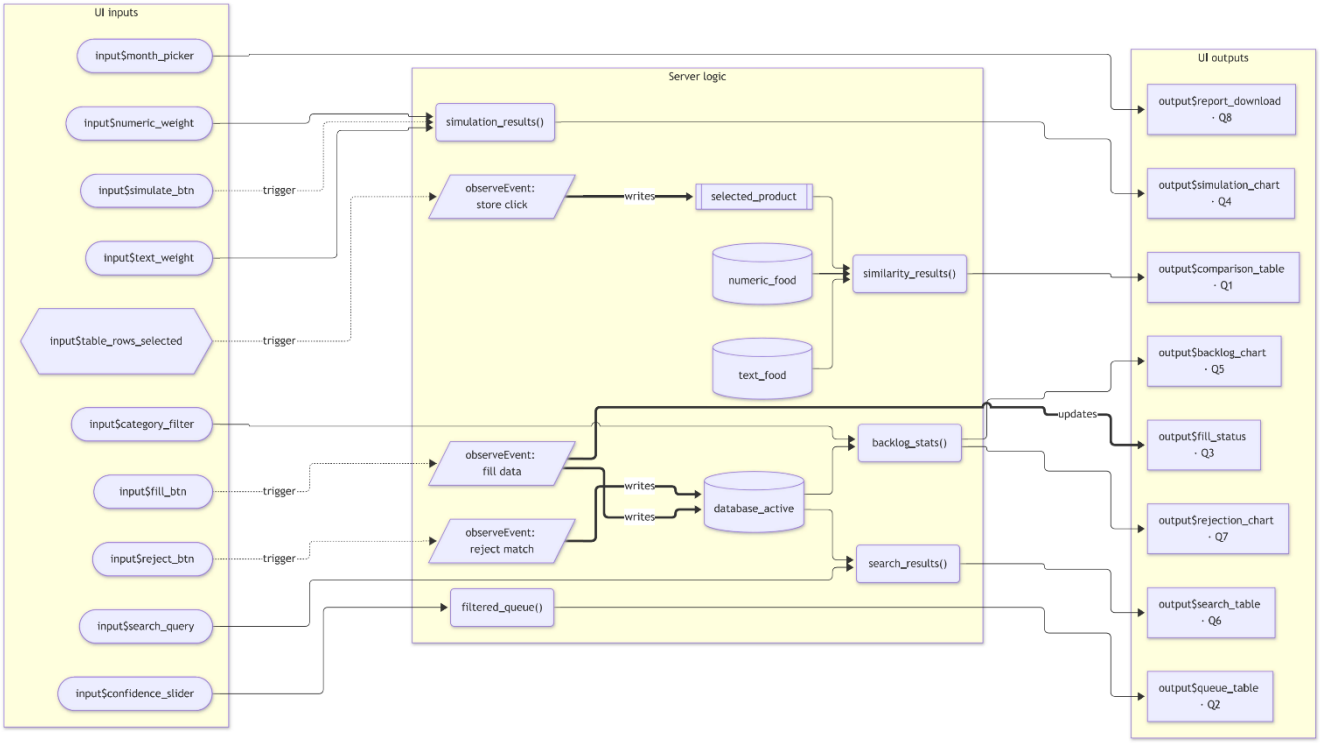
- **Observers (Side effects):**
    - **`observeEvent(input$table_rows_selected, ...)`:** This listens for a click on the queue table and **writes** to selected_product (the State). The rule we applied here is *"Store selections as state"*.
    - **`observeEvent(input$fill_btn, ...)`:** Updates the database with the new values.
- **Outputs (Views):**
    - `output$comparison_table` (Q1)
    - `output$queue_table` (Q2)
    - `output$simulation_chart` (Q4)
    - `output$backlog_chart` (Q5)
    - `output$search_table` (Q6)
    - `output$rejection_chart` (Q7)
    - `output$report_download` (Q8)

The Mermaid Diagram

The Mermaid diagram is the visual blueprint of the Reactive Graph. It is built strictly using the shape conventions from the course slides:

- **cylinders** for **data**,
- **stadiums** for **inputs**,
- **hexagons** for **events**,
- **subroutines** for **state**,
- **rounded boxes** for **conductors**,
- **parallelograms** for **observers**,
- and **rectangles** for **outputs**.

The arrows map exactly how data flows from the user's click to the final chart.



Reactive Graph Design & Rationale

- **Compute once, use many times:** We use a `reactive()` called `similarity_results()` to calculate the matches. This single calculation feeds the comparison table (Q1) and provides the data needed for the "Fill Missing Data" action (Q3).
- **Separate cheap and expensive work:** The simulation (Q4) is computationally heavy. We use an `eventReactive()` so it only runs when the developer clicks the "Simulate" button, rather than reacting to every slider movement.
- **Store selections as state:** When a dietitian clicks a row in the queue table (Q1), we use an `observeEvent()` to save that product into a `reactiveVal()` called `selected_product`. This ensures the selection doesn't vanish if the table refreshes.
- **Guard empty states:** The comparison table (Q1) will use `req(selected_product())` so that the app doesn't crash before a product is clicked.